



CI1056 – ALGORITMOS E ESTRUTURA DE DADOS II

Profa. Rachel Reis
rachel@inf.ufpr.br



AULA DE HOJE

Remover Recursão

Adaptação do material gentilmente
disponibilizado pelos Profs. André Vignatti e
Daniel Oliveira

MOTIVAÇÃO

- Como funciona a chamada de funções:
 - Chamadas de funções, recursivas ou não, são implementadas por uma pilha.
 - Na pilha, geralmente, temos:
 - Estado anterior: informações para retornar exatamente de onde o programa efetuou a chamada.
 - Parâmetros: valores passados como argumentos.
 - Variáveis locais: são instanciadas na pilha ao invés da *'heap'* do sistema por questões de eficiência.

MOTIVAÇÃO

- Podemos evitar usar a pilha gerenciada pelo S.O., a partir da criação de algoritmos iterativos que não usam essa pilha.
- Para alguns casos/linguagens, um ganho de espaço e tempo de execução pode ser obtido ao usar algoritmos iterativos.

REMOVENDO A RECURSÃO

- Vamos ver duas formas de retirar a recursão de um algoritmo:
 - Removendo a recursão de cauda
 - Usando pilha criada pelo programador (não gerenciada pelo S.O.).

RECURSÃO DE CAUDA

- Chamadas recursivas feitas apenas no final da função
 - Não há nenhuma operação após alguma chamada recursiva

funcao1(a, b)

...;
funcao1(x, y);

funcao2(a, b)

...;
if $a > b$ **then**
...;
 funcao2(x, y);
else
 funcao2(w, z);

REMOVENDO A RECURSÃO DE CAUDA

- Seguindo o **padrão** abaixo, podemos traduzir um algoritmo recursivo (de cauda) em iterativo.

Recursão de cauda

$\text{funcao1}(a, b)$

if *condição_parada* **then**
 return R

else

 Bloco de operações;

return $\text{funcao1}(x, y)$;

Versão iterativa

$\text{funcao1_iterativa}(a, b)$

while *Não condição_parada* **do**
 Bloco de operações;

$a \leftarrow x$;

$b \leftarrow y$;

return R

REMOVENDO A RECURSÃO DE CAUDA

- Passo 1: colocar a negação da **condição de parada** da versão recursiva, na condição do laço de repetição da versão iterativa.

Recursão de cauda

$\text{funcao1}(a, b)$

if condição_parada then
 return R

else

 Bloco de operações;

 return $\text{funcao1}(x, y)$;

Versão iterativa

$\text{funcao1_iterativa}(a, b)$

while Não condição_parada do

 Bloco de operações;

$a \leftarrow x$;

$b \leftarrow y$;

return R

REMOVENDO A RECURSÃO DE CAUDA

- Passo 2: colocar o **bloco de operações** da versão recursiva, no início do corpo do laço de repetição da versão iterativa.

Recursão de cauda

```
funcao1(a, b)
```

```
  if condição_parada then  
    return R
```

```
  else
```

```
    Bloco de operações;
```

```
    return funcao1(x, y);
```

Versão iterativa

```
funcao1_iterativa(a, b)
```

```
  while Não condição_parada do
```

```
    Bloco de operações;
```

```
     $a \leftarrow x$ ;
```

```
     $b \leftarrow y$ ;
```

```
  return R
```

REMOVENDO A RECURSÃO DE CAUDA

- Passo 3: transformar os argumentos da chamada recursiva em operações de atribuição na versão iterativa.

Recursão de cauda

$\text{funcao1}(a, b)$

if *condição_parada* **then**
 return R

else

 Bloco de operações;

return $\text{funcao1}(x, y);$

Versão iterativa

$\text{funcao1_iterativa}(a, b)$

while *Não condição_parada* **do**

 Bloco de operações;

$a \leftarrow x;$

$b \leftarrow y;$

return R

REMOVENDO A RECURSÃO DE CAUDA

- Passo 4: ao finalizar o laço de repetição da versão iterativa temos que retornar o valor da condição de parada (versão recursiva).

Recursão de cauda

funcao1(a, b)

if condição_parada then

return R

else

Bloco de operações;

return funcao1(x, y);

Versão iterativa

funcao1_iterativa(a, b)

while Não condição_parada do

Bloco de operações;

a ← x;

b ← y;

return R

EXEMPLO 1- FIBONACCI

- Fibonacci normalmente não é recursão de cauda.

$\text{Fibonacci}(n)$

if $n \leq 1$ then
 return n

else

 return $\text{Fibonacci}(n - 1) + \text{Fibonacci}(n - 2);$

- Mas fibonacci pode ser **modificado** para que a recursão se torne de **cauda**.

0	1	2	3	4	5	6	7
↓	↓	↓	↓	↓	↓	↓	↓

0, 1, 1, 2, 3, 5, 8, 13, ...

Fib(4, 0, 1) = 3

$x \leftarrow 1$

$y \leftarrow 1$

Fib(3, 1, 1)

$x \leftarrow 1$

$y \leftarrow 2$

Fib(2, 1, 2)

$x \leftarrow 2$

$y \leftarrow 3$

Fib(1, 2, 3)

Fibonacci(n, a, b)

if $n = 0$ **then**

return a

if $n = 1$ **then**

return b

else

$x \leftarrow b$;

$y \leftarrow a + b$;

return $\text{Fibonacci}(n - 1, x, y)$

- Passo 1: colocar a negação das **condições de parada** da versão recursiva, na condição do laço de repetição da versão iterativa.

$$\text{Fibonacci}(n, a, b)$$

if $n = 0$ then

 return a

if $n = 1$ then

 return b

else

$x \leftarrow b;$

$y \leftarrow a + b;$

 return $\text{Fibonacci}(n - 1, x, y)$

$$\text{Fibonacci}(n, a, b)$$

while $n > 1$ do

- Passo 2: colocar o **bloco de operações** da versão recursiva, no início do corpo do laço de repetição da versão iterativa.

$$\text{Fibonacci}(n, a, b)$$

if $n = 0$ then

 return a

if $n = 1$ then

 return b

else

$x \leftarrow b;$

$y \leftarrow a + b;$

 return $\text{Fibonacci}(n - 1, x, y)$

$$\text{Fibonacci}(n, a, b)$$

while $n > 1$ do

$x \leftarrow b;$

$y \leftarrow a + b;$

- Passo 3: transformar a chamada recursiva em operações de atribuição.

$\text{Fibonacci}(n, a, b)$

if $n = 0$ **then**

return a

if $n = 1$ **then**

return b

else

$x \leftarrow b;$

$y \leftarrow a + b;$

return $\text{Fibonacci}(n - 1, x, y)$

$\text{Fibonacci}(n, a, b)$

while $n > 1$ **do**

$x \leftarrow b;$

$y \leftarrow a + b;$

$a \leftarrow x;$

$b \leftarrow y;$

$n \leftarrow n - 1;$

- Passo 4: ao finalizar o laço de repetição da versão iterativa temos que retornar os valores das condições de parada (versão recursiva).

$\text{Fibonacci}(n, a, b)$

if $n = 0$ **then**
 return a
if $n = 1$ **then**
 return b

else

$x \leftarrow b;$

$y \leftarrow a + b;$

return $\text{Fibonacci}(n - 1, x, y)$

$\text{Fibonacci}(n, a, b)$

while $n > 1$ **do**

$x \leftarrow b;$

$y \leftarrow a + b;$

$a \leftarrow x;$

$b \leftarrow y;$

$n \leftarrow n - 1;$

if $n = 0$ **then**
 return a
if $n = 1$ **then**
 return b

EXEMPLO 1- FIBONACCI

Recursão de cauda

$\text{Fibonacci}(n, a, b)$

if $n = 0$ **then**

return a

if $n = 1$ **then**

return b

else

$x \leftarrow b;$

$y \leftarrow a + b;$

return $\text{Fibonacci}(n - 1, x, y)$

Versão iterativa

$\text{Fibonacci}(n, a, b)$

while $n > 1$ **do**

$x \leftarrow b;$

$y \leftarrow a + b;$

$a \leftarrow x;$

$b \leftarrow y;$

$n \leftarrow n - 1;$

if $n = 0$ **then**

return a

if $n = 1$ **then**

return b

EXEMPLO 2- BUSCA BINÁRIA

- O algoritmo de busca binária utiliza **recursão de cauda**?

BuscaBinaria(x, v, a, b)

if $a > b$ **then**

return *NÃO*

$m \leftarrow \lfloor \frac{a+b}{2} \rfloor$;

if $v[m] = x$ **then**

return m

if $x < v[m]$ **then**

return *BuscaBinaria*($x, v, a, m - 1$)

else

return *BuscaBinaria*($x, v, m + 1, b$)

- Passo 1: colocar a negação da **condição de parada** da versão recursiva, no laço de repetição da versão iterativa.

$\text{BuscaBinaria}(x, v, a, b)$

if $a > b$ then

 return $N\tilde{A}O$

$m \leftarrow \lfloor \frac{a+b}{2} \rfloor$;

if $v[m] = x$ then

 return m

if $x < v[m]$ then

 return $\text{BuscaBinaria}(x, v, a, m - 1)$

else

 return $\text{BuscaBinaria}(x, v, m + 1, b)$

$\text{BuscaBinaria}(x, v, a, b)$

while $a \leq b$ do

- Passo 2: colocar o **bloco de operações** da versão recursiva, no corpo do laço de repetição da versão iterativa.

BuscaBinaria(x, v, a, b)

if $a > b$ then

return *NÃO*

$m \leftarrow \lfloor \frac{a+b}{2} \rfloor$;

if $v[m] = x$ then

return m

if $x < v[m]$ then

return *BuscaBinaria*($x, v, a, m - 1$)

else

return *BuscaBinaria*($x, v, m + 1, b$)

BuscaBinaria(x, v, a, b)

while $a \leq b$ do

$m \leftarrow \lfloor \frac{a+b}{2} \rfloor$;

if $v[m] = x$ then

return m

if $x < v[m]$ then

else

- Passo 3: transformar os argumentos das chamadas recursivas em operações de atribuição.

$\text{BuscaBinaria}(x, v, a, b)$

if $a > b$ **then**

return $N\tilde{A}O$

$m \leftarrow \lfloor \frac{a+b}{2} \rfloor$;

if $v[m] = x$ **then**

return m

if $x < v[m]$ **then**

return $\text{BuscaBinaria}(x, v, a, m - 1)$

else

return $\text{BuscaBinaria}(x, v, m + 1, b)$

$\text{BuscaBinaria}(x, v, a, b)$

while $a \leq b$ **do**

$m \leftarrow \lfloor \frac{a+b}{2} \rfloor$;

if $v[m] = x$ **then**

return m

if $x < v[m]$ **then**

$b = m - 1$;

else

$a = m + 1$;

- Passo 4: ao finalizar o laço de repetição da versão iterativa temos que retornar os valores das condições de parada (versão recursiva).

$\text{BuscaBinaria}(x, v, a, b)$

if $a > b$ **then**

return $N\tilde{A}O$

$m \leftarrow \lfloor \frac{a+b}{2} \rfloor$;

if $v[m] = x$ **then**

return m

if $x < v[m]$ **then**

return $\text{BuscaBinaria}(x, v, a, m - 1)$

else

return $\text{BuscaBinaria}(x, v, m + 1, b)$

$\text{BuscaBinaria}(x, v, a, b)$

while $a \leq b$ **do**

$m \leftarrow \lfloor \frac{a+b}{2} \rfloor$;

if $v[m] = x$ **then**

return m

if $x < v[m]$ **then**

$b = m - 1$;

else

$a = m + 1$;

return $N\tilde{A}O$

EXEMPLO 2- BUSCA BINÁRIA

Recursão de cauda

$\text{BuscaBinaria}(x, v, a, b)$

```
if  $a > b$  then
    return  $N\tilde{A}O$ 
 $m \leftarrow \lfloor \frac{a+b}{2} \rfloor$ ;
if  $v[m] = x$  then
    return  $m$ 
if  $x < v[m]$  then
    return  $\text{BuscaBinaria}(x, v, a, m - 1)$ 
else
    return  $\text{BuscaBinaria}(x, v, m + 1, b)$ 
```

Versão iterativa

$\text{BuscaBinaria}(x, v, a, b)$

```
while  $a \leq b$  do
     $m \leftarrow \lfloor \frac{a+b}{2} \rfloor$ ;
    if  $v[m] = x$  then
        return  $m$ 
    if  $x < v[m]$  then
         $b = m - 1$ ;
    else
         $a = m + 1$ ;
return  $N\tilde{A}O$ 
```

REMOVER A RECURSÃO - RESUMO

- Para utilizar a remoção da recursão usando recursão de cauda
 - 1) Verificar se o algoritmo utiliza recursão de cauda.
 - 1.1 Se utilizar, traduzir o algoritmo para a versão iterativa.
 - 1.2 Se não utilizar...
 - 1.2.1 Verificar se o algoritmo recursivo pode ser transformado para recursão de cauda.
 - 1.2.1.1 Se puder, traduzir o algoritmo para a versão iterativa.
 - 1.2.1.2 Se não puder, usar pilhas.

PILHA GERENCIADA PELO S.O.

- Ideia de pilha de chamadas controlada pelo S.O.:
 - Ao chamar uma função: empilha as informações necessárias para executá-la.
 - Ao retornar uma função: desempilha as informações e retorna para a função que está no topo da pilha.

PILHA CONTROLADA PELO PROGRAMADOR

- Padrão que funciona na maioria das vezes:
 - Chamadas recursivas são substituídas por empilha.
 - Retorno de funções são substituídas por desempilha.
 - Esse processo (empilha, desempilha) será realizado dentro de um laço até que a pilha fique vazia.

EXEMPLO - QUICKSORT

Algoritmo recursivo

Quicksort(v, a, b)

Se $a \geq b$

 Devolva v

Senão

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

 Quicksort($v, a, m - 1$)

 Quicksort($v, m + 1, b$)

Algoritmo iterativo (pilha)

Quicksort_i(v, a, b)

iniciaPilha(p)

empilha(p, a, b)

Enquanto p não está vazia

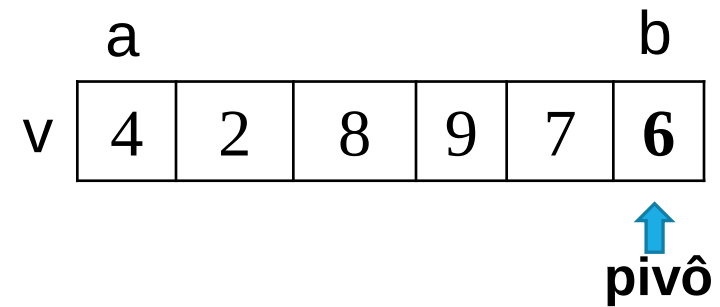
$a, b \leftarrow \text{desempilha}(p)$

 Se $a < b$

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

 empilha($p, a, m - 1$)

 empilha($p, m + 1, b$)



Algoritmo iterativo

Quicksort_i(v, a, b)

iniciaPilha(p)

empilha(p, a, b)

Enquanto p não está vazia

$a, b \leftarrow$ desempilha(p)

 Se $a < b$

$m \leftarrow$ Particiona($v, a, b, v[b]$)

 empilha($p, a, m - 1$)

 empilha($p, m + 1, b$)

PEGADINHA 1

ORDEM DE EXECUÇÃO

- Exemplo: a **ordem** em que os algoritmos (recursivo e iterativo) fazem a ordenação dos subvetores é **diferente**.

Algoritmo recursivo

Quicksort(v, a, b)

Se $a \geq b$

 Devolva v

Senão

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

 Quicksort($v, a, m - 1$)

 Quicksort($v, m + 1, b$)

Algoritmo iterativo (pilha)

Quicksort_i(v, a, b)

iniciaPilha(p)

empilha(p, a, b)

Enquanto p não está vazia

$a, b \leftarrow \text{desempilha}(p)$

 Se $a < b$

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

 empilha($p, a, m - 1$)

 empilha($p, m + 1, b$)

PEGADINHA 1

ORDEM DE EXECUÇÃO

- Múltiplas chamadas recursivas serão executadas na ordem inversa da versão iterativa.
 - Caso a ordem de execução seja importante, devemos empilhar na ordem inversa que acontecem as chamadas recursivas.

PEGADINHA 1

ORDEM DE EXECUÇÃO

- Exemplo

Versão recursiva

$\text{Funcao}(a, b)$

...

$\text{Funcao}(a + 1, b)$

$\text{Funcao}(a, b + 1)$

Versão iterativa 1

$\text{Funcao_i1}(a, b)$

...

Enquanto p não está vazia
 $a, b \leftarrow \text{desempilha}(p)$

...

$\text{empilha}(p, a + 1, b)$

$\text{empilha}(p, a, b + 1)$

Versão iterativa 2

$\text{Funcao_i2}(a, b)$

...

Enquanto p não está vazia
 $a, b \leftarrow \text{desempilha}(p)$

...

$\text{empilha}(p, a, b + 1)$

$\text{empilha}(p, a + 1, b)$

PEGADINHA 2

CONTINUIDADE

- Ao empilhar os dados de uma nova chamada recursiva a execução atual deve parar.

Versão recursiva

Contador(n)

Se $n = 1$

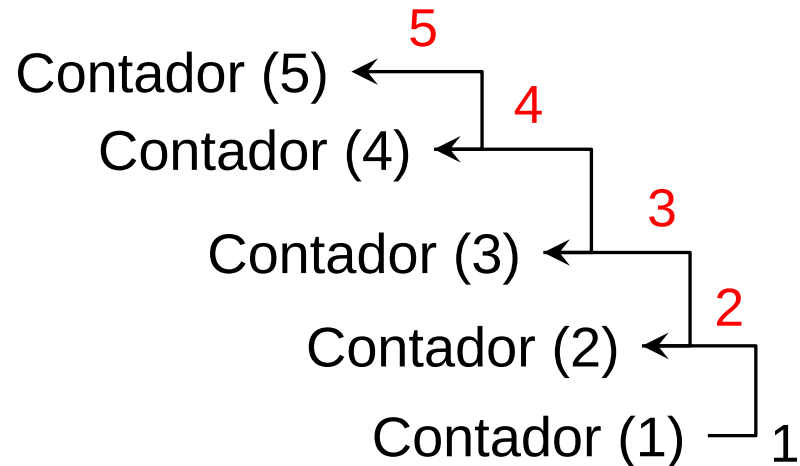
Imprime n

Senão

→ Contador($n - 1$)

Imprime n

Saída: 1, 2, 3, 4, 5



PEGADINHA 2

CONTINUIDADE

- Problema: na versão iterativa, o programa executa a função imprime antes de processar os dados na pilha.

Versão recursiva

Contador(n)

Se $n = 1$

Imprime n

Senão

Contador($n - 1$)

Imprime n

← 1, 2, 3, 4, 5

5, 4, 3, 2, 1 →

Versão iterativa

Contador(n)

iniciaPilha(p)

empilha(p, n)

Enquanto p não está vazia

$n \leftarrow$ desempilha(p)

Se $n = 1$

Imprime n

Senão

empilha($p, n - 1$)

Imprime n

- Solução usando switch: o switch pode ser usado para **controlar** em que **linha** o programa parou na hora da chamada.

Versão iterativa 1

Contador(n)

```
iniciaPilha( $p$ )
empilha( $p, n$ )
Enquanto  $p$  não está vazia
     $n \leftarrow desempilha(p)$ 
    Se  $n = 1$ 
        Imprime  $n$ 
    Senão
        empilha( $p, n - 1$ )
        Imprime  $n$ 
```

Versão iterativa 2

```
iniciaPilha( $p$ )
empilha( $p, n, A$ )
Enquanto  $p$  não está vazia
     $n, linha = desempilha(p)$ 
    switch( $linha$ ) {
        case 'A':
            Se  $n = 1$ 
                Imprime  $n$ 
            Senão
                empilha( $p, n, B$ )
                empilha( $p, n-1, A$ )
        case 'B':
            Imprime  $n$ 
```

CONCLUSÃO

- Todo algoritmo recursivo pode ser transformado em iterativo.
- Remover a recursão pode resultar em ganho de desempenho (espaço em memória e tempo de execução).
- E por que fazer isso?
 - Algumas linguagens/dispositivos **não** permitem recursão.